

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №9

по теме:

ПРИЕМ QPSK VKSK, ПРИЕМ НА СОГЛАСОВАННЫЙ ФИЛЬТР,
ГЛАЗКОВАЯ ДИАГРАММА, ПОИСК ОПТИМАЛЬНОГО ОТСЧЕТНОГО
ЗНАЧЕНИЯ И НЕОБХОДИМОСТЬ СИМВОЛЬНОЙ синхронизации.
ПРИЕМ И ФИЛЬТРАЦИЯ СИГНАЛА. ПРЯМОУГОЛЬНЫЙ И
ПРИПОДНЯТЫЙ КОСИНУС

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ЛЕКЦИЯ	4
2.1	Архитектура приемника	4
2.1.1	Радиоканал и SDR	4
2.1.2	Matched filter	4
2.1.3	Downsampling	5
2.1.4	Quantization	6
2.1.5	Demapper	7
2.1.6	Decoder	7
3	ПРАКТИКА	8
3.1	Match filter	8
3.2	Downsampling	8
3.3	downscale	9
3.4	Quantization	10
3.5	Demapper	11
3.6	Влияние сдвига на созвездие	13
3.7	Тест	13
3.8	Результат работы	15
4	ВЫВОД	16

ВВЕДЕНИЕ

1.1 Введение

До этого занятия мы рассматривали логику работы передатчика. Мы узнали, как передатчик формирует сигнал, как осуществить отправку данных с SDR с помощью C/C++ и библиотеки SoapySDR. С этого занятия будет рассматриваться приемник. В ходе этих занятий мы реализуем логику работы приемной стороны, символьную синхронизацию, чтобы извлечь из принятых семплов передаваемые биты. Конкретно на этом занятии реализуем логику по обработке принимаемого сигнала без синхронизации.

1.2 Цель

Реализовать логику работы применой стороны на языке C/C++. Выяснить, как выбор семпла в качестве значения символа влияет на созвездие. Убедиться в необходимости символьной синхронизации.

ЛЕКЦИЯ

2.1 Архитектура приемника

Рассмотрим архитектуру простого приемника

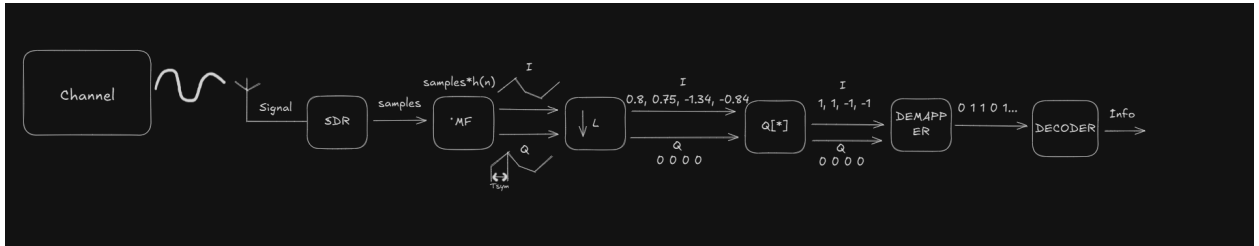


Рисунок 1 — Архитектура простого приемника

2.1.1 Радиоканал и SDR

Из радиоканала сигнал попадает на антенну, где преобразуется в электрический сигнал и попадает в SDR, которая дискретизирует сигнал, производит какие-то дополнительные манипуляции и на выходе выдает отсчеты сигнала.

2.1.2 Matched filter

Далее семплы попадают на согласованный фильтр (matched filter), который позволяет увеличить SNR (Signal Noise Ration), т.е выделить полезный сигнал из общего сигнала с шумом. Импульсная характеристика фильтра должна согласовываться с импульсной характеристикой фильтра в передатчике. Допустим, передатчик имеет импульсную характеристику $g(n)$, тогда на приемной стороне ИХ фильтра будет следующей:

$$h(n) = Kg * (L - n)$$

ИХ согласованного фильтра на приемнике - ИХ, которая является комплексно-сопряженной (зеркальной) с ИХ передатчика и сдвинутой на L отсчетов, где L - кол-во семплов на символ. K - коэффициент.

Мы используем прямоугольную ИХ на передатчике, поэтому на приеме эта ИХ никак не изменится.

На выходе фильтра получаем пилообразный график:

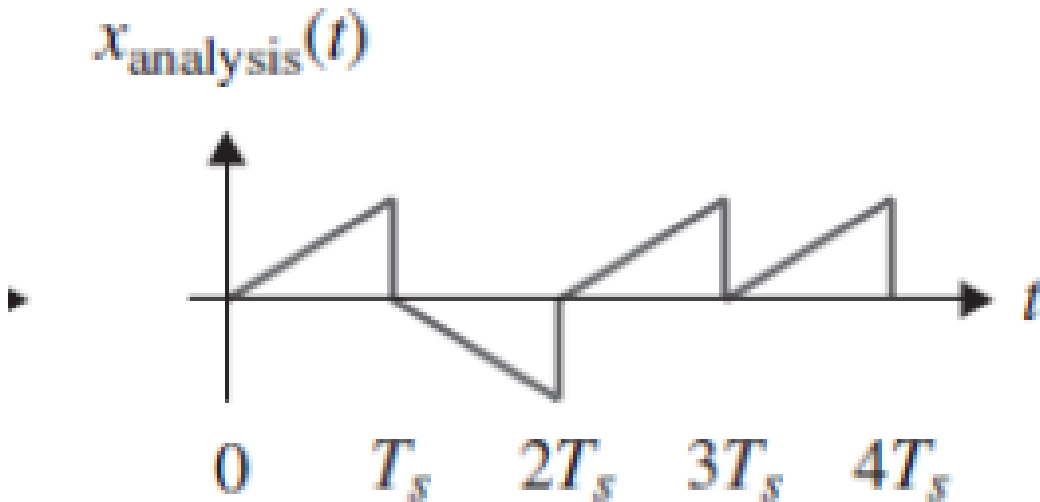


Рисунок 2 — Выход matched filter

2.1.3 Downsampling

На стороне передатчика мы делали операцию upsampling, которая повышала частоту дискретизации. Это позволяло нам сделать так, чтобы на 1 символ приходилось 10 семплов. На стороне приема делается обратная операция - downsampling, т.е нам нужно из каждых 10 семплов выбрать один такой, который наиболее точно описывает переданный символ. Но какой семпл нужно выбирать? После прохождения через фильтр каждый 10-ый семпл будет иметь наибольший SNR, именно его и нужно выбирать. Но с какого семпла нам нужно начинать выполнять эту операцию? Ведь мы можем "промахнуться" и начать выбирать не те семплы. На этот вопрос отвечает символьная синхронизация, но это тема последующих занятий. На этом занятии будет начинаться downsampling с первого семпла.

2.1.4 Quantization

Проходя через радиоканал, сигнал искажается и принятые символы будут искаженными. Если на передаче отправлялись символы $\{1,0,1,0,-1,0\}$, то на приеме можем получить $\{0.74,-0.32,1.24,0.2,-1.35,0.01\}$. Четко определить, какой последовательности битов соответствуют символы невозможно, поэтому перед тем, как начать этап преобразования символов в биты, нужно понять, к какой из возможных точек созвездия эти точки находятся ближе всего, и как-бы округлить координаты точек так, чтобы они точно соответствовали координатам истинных точек созвездия. Чтобы оценивать, к какой истинной точке на созвездии ближе всего находится точка с искаженными координатами, я буду вычислять расстояние между искаженной точкой и всеми истинными точками.

Пример вычисления расстояния между точками сигнального созвездия BPSK:

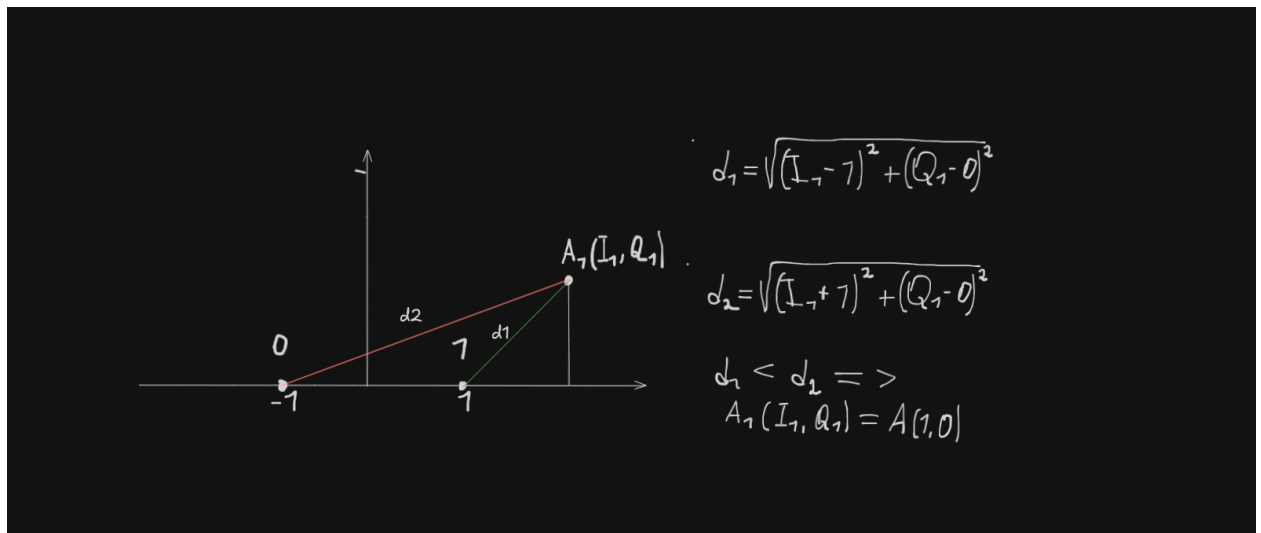


Рисунок 3 — Расстояние между точками созвездия

В созвездии BPSK имеются 2 точки с координатами $(-1,0)$ и $(1,0)$, точка A имеет искаженные координаты. По теореме Пифагора можно найти расстояния между точками. Точка A ближе находится к точке $(1,0)$, поэтому логично предположить, что передавалась единица, а не ноль.

2.1.5 Demapper

На передаче мы подавали биты на специальное устройство под названием Mapper, которое ставило в соответствие последовательности бит символы. Demapper делает обратную задачу - символам сопоставляет биты.

2.1.6 Decoder

На стороне передатчика передается текст, поэтому на стороне приема нужен блок, который преобразовывал бы биты в текст. Этой задачей будет заниматься блок Decoder.

ПРАКТИКА

3.1 Match filter

```
int16_t* match_filter(int16_t* samples, int samples_count, int
    L, double* h, int* size){
    *size = samples_count;
    int16_t* new_samples = (int16_t*)malloc(samples_count *
        sizeof(int16_t));

    //convolve
    for(int n = 0; n < samples_count; ++n){
        int16_t tmp = 0;
        for(int m = 0; m < L; ++m){
            if (n - m >= 0){
                tmp += samples[n-m]*h[m];
            }
        }

        new_samples[n] = tmp;
    }

    return new_samples;
}
```

Функция принимает массив семплов, размер этого массива, число L , которое показывает, сколько семплов приходится на бит, ИХ h , переменную $size$, в которой вернется размер массива на выходе. Функция выполняет дискретную свертку входного сигнала с ИХ.

3.2 Downsampling

```
int16_t* down_sampling(int16_t* samples, int samples_count, int
    L, int start){
    int symbols_size = samples_count / L - start + 1;

    int16_t* symbols = (int16_t*)malloc(symbols_size *
        sizeof(int16_t));
}
```



```

    for(int i = start; i < symbols_size; ++i){
        symbols[i] = samples[i*L];
    }

    return symbols;
}

```

Функция принимает массив исходных семплов, размер массива, число L , которое указывает на то, с каким периодом мы будем делать downsampling (сохраняем только каждое L -ое значение). Параметр `start` - индекс, с которого функция начнет брать каждый L -ый элемент. В функции итерируемся по новому массиву и заполняем его каждым L -ым значением.

3.3 downscale

```

double max(int16_t* arr, int arr_size){
    double max = -__DBL_MAX__;
    for(int i = 0; i < arr_size; ++i){
        if(arr[i] > max){
            max = arr[i];
        }
    }

    return max;
}

double* downscale(int16_t* samples, int samples_count){
    double* down_scale_samples = (double*)malloc(samples_count *
        sizeof(double));

    double max_value = max(samples, samples_count);

    for(int i = 0; i < samples_count; i+=2){
        down_scale_samples[i] = samples[i] / max_value;
    }

    return down_scale_samples;
}

```

Для того, чтобы дальше работать с символами, нужно нормализовать их, т.е привести к значениям, которые будут близки к значениям из отрезка $[-1;1]$. В качестве нормирующего коэффициента будет использоваться символ с наибольшим значением. Функция `max` используется для поиска символа с максимальным значением. В функции `downscale` итерируемся по исходному массиву символов и делим каждый на максимальное значение среди символов.

3.4 Quantization

```
int16_t* BPSK_quantizater(double* symbols, int symbols_size){
    int16_t* rounded_symbols = (int16_t*)malloc(symbols_size *
        sizeof(int16_t));

    int point0[] = {-1, 0};
    int point1[] = {1, 0};

    double distance0 = 0, distance1 = 0;

    double dx = 0, dy = 0;

    for(int i = 0; i < symbols_size; i+=2){
        dx = point0[0]-symbols[i];
        dy = point0[1]-symbols[i+1];

        distance0 = sqrt(dx*dx + dy*dy);

        dx = point1[0]-symbols[i];
        dy = point1[1]-symbols[i+1];

        distance1 = sqrt(dx*dx + dy*dy);

        if(distance0 > distance1){
            rounded_symbols[i] = 1;
            rounded_symbols[i+1] = 0;
        }else{
            rounded_symbols[i] = -1;
            rounded_symbols[i+1] = 0;
        }
    }
}
```

```

        return rounded_symbols;
    }

```

Функция принимает массив искаженных символов и размер массива. В функции в виде массивов размером в 2 элемента задаются истинные точки, которые есть в BPSK созвездии. Далее в цикле перебираем символы (точки на созвездии) и вычисляем расстояние между искаженной точкой и заданными истинными точками. В результирующий массив запишем координаты истинной точки, с которой искаженная точка имеет наименьшее расстояние.

3.5 Demapper

```

int8_t* BPSK_demodulator(int16_t* symbols, int symbols_size){
    int bits_size = symbols_size/2;

    int8_t* bits = (int8_t*)malloc(bits_size * sizeof(int8_t));

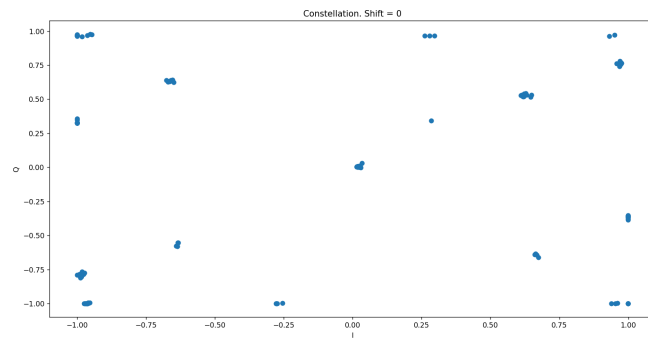
    int j = 0;

    for(int i = 0; i < symbols_size; i+=2){
        if(symbols[i] == 1){
            bits[j++] = 1;
        } else{
            bits[j++] = 0;
        }
    }

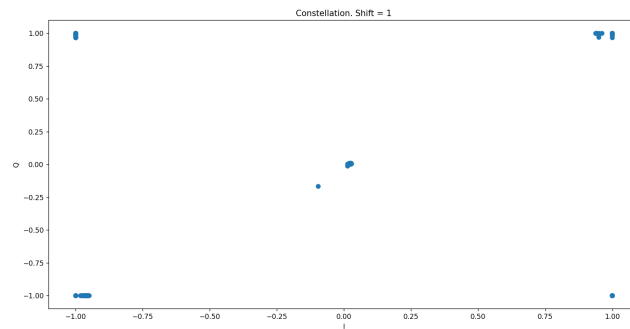
    return bits;
}

```

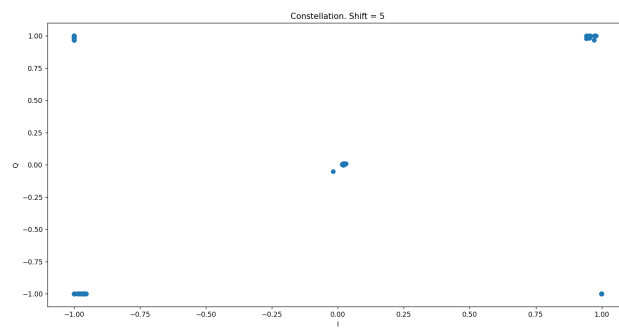
Функция принимает массив округленных символов и размер массива. Функция итерируется по массиву и с помощью условий выбирает, какой бит сопоставить символу.



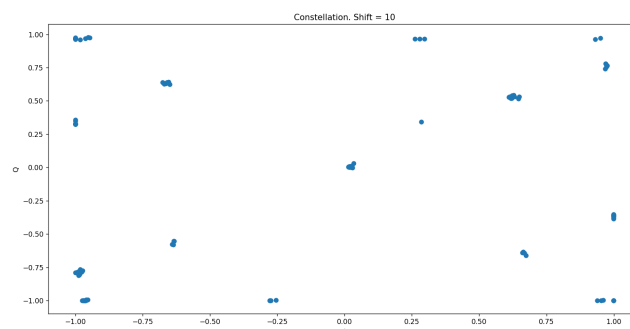
(a) Shift = 0



(б) Shift = 1



(в) Shift = 5



(г) Shift = 10

Рисунок 4 — Влияние сдвига семплов на созвездие

3.6 Влияние сдвига на созвездие

Можем заметить, что при нулевом сдвиге созвездие ”распалось т.е мы начали брать отсчеты неправильно. При сдвиге равном 1 созвездие ”собралось” во что-то похожее на QPSK созвездие. При сдвиге равном 5 созвездие стало еще более похожим на QPSK созвездие. На сдвиге равном 10 созвездие снова ”распалось”. Этот процесс будет повторяться периодически на каждом символе. Символьная синхронизация позволяет автоматизировать этот процесс и подобрать оптимальный момент для начала взятия отсчетов.

3.7 Тест

Для проверки корректности работы приемника сгенерируем семплы на стороне передатчика и запишем в файл, а на приемной стороне считаем файл и попытаемся превратить семплы в информацию, которую отправлял передатчик. Попытаемся передать символ ”Н”

```
int main(){

    size_t samples_count = 0;

    int16_t* samples = read_pcm("bpsk_samples.pcm",
        &samples_count);

    if(samples == nullptr){
        printf("File not found!\n");
        exit(1);
    }

    printf("samples:\n");

    for(int i = 0; i < samples_count; ++i){
        printf("%d ", samples[i]);
    }

    printf("\n");

    int h[] = {1,1,1,1,1,1,1,1,1,1};

    int conv_size = 0;
```

```

int* conv = match_filter(samples, samples_count, 10, h,
    &conv_size);

printf("After convolve:\n");

for(int i = 0; i < conv_size; ++i){
    printf("%d ", conv[i]);
}

printf("\n");

int* ds_samples = down_sampling(conv, samples_count, 10, 0);

printf("Samples after down sampling:\n");

for(int i = 0; i < conv_size/10; ++i){
    printf("%d ", ds_samples[i]);
}

printf("\n");

double* down_scale_samples = down_scaler(ds_samples,
    samples_count/10);

printf("Samples after downscaling:\n");

for(int i = 0; i < conv_size/10; ++i){
    printf("%f ", down_scale_samples[i]);
}

printf("\n");

int16_t* q_samples = BPSK_quantizater(down_scale_samples,
    samples_count/10);

printf("rounded symbols:\n");

for(int i = 0; i < samples_count/10; ++i){
    printf("%d ", q_samples[i]);
}

```

```

printf("\n");

int8_t* bits1 = BPSK_demodulator(q_samples,
    samples_count/10);

printf("bits:\n");

for(int i = 0; i < samples_count/20; ++i){
    printf("%d ", bits1[i]);
}

printf("\n");

int str_len = 0;
char* str = btos(bits1, samples_count/20, &str_len);

printf("string: %s\n", str);

return 0;
}

```

3.8 Результат работы

```

file_size = 320

samples:
-32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 0 32752 32752 32752 32752 32752 32752 32752 32752 32752 0 0 0 0 0 0 0 0 -32752
-32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 0 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 32752
32752 32752 32752 32752 32752 32752 32752 32752 32752 0 0 0 0 0 0 0 0 0 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 -32752 -32752
-32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 0 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 -32752 0 0 0 0 0 0 0 0 -32752 -32752
After convolve:
-32752 -65504 -98256 -131008 -163760 -196512 -229264 -262016 -294768 -327520 -294768 -262016 -229264 -196512 -163760 -131008 -98256 -65504 -32752 0 32752 65504 98256 131008 163760
196512 229264 262016 294768 327520 294768 262016 229264 196512 163760 131008 98256 65504 32752 0 -32752 -65504 -98256 -131008 -163760 -196512 -229264 -262016 -294768 -327520 -294768
-262016 -229264 -196512 -163760 -131008 -98256 -65504 -32752 0 -32752 -65504 -98256 -131008 -163760 -196512 -229264 -262016 -294768 -327520 -294768 -262016 -229264 -196512 -163760
-131008 -98256 -65504 -32752 0 32752 65504 98256 131008 163760 196512 229264 262016 294768 327520 294768 262016 229264 196512 163760 131008 98256 65504 32752 0 -32752 -65504 -98256
-131008 -163760 -196512 -229264 -262016 -294768 -327520 -294768 -262016 -229264 -196512 -163760 -131008 -98256 -65504 -32752 0 -32752 -65504 -98256 -131008 -163760 -196512 -229264
-262016 -294768 -327520 -294768 -262016 -229264 -196512 -163760 -131008 -98256 -65504 -32752 0 -32752 -65504 -98256 -131008 -163760 -196512 -229264 -262016 -294768 -327520 -294768
-262016 -229264 -196512 -163760 -131008 -98256 -65504 -32752 0
Samples after down sampling:
-32752 -294768 32752 294768 -32752 -294768 32752 294768 -32752 -294768 32752 -294768 -32752 -294768 32752 -294768
Samples after downscaling:
-0.111111 -1.000000 0.111111 1.000000 -0.111111 -1.000000 -0.111111 -1.000000 0.111111 1.000000 -0.111111 -1.000000 -0.111111 -1.000000 -0.111111 -1.000000
rounded symbols:
-1 0 1 0 -1 0 -1 0 1 0 -1 0 -1 0 -1 0
bits:
0 1 0 0 1 0 0 0
string: H

```

Рисунок 5 — Процесс обработки принятого сигнала

Приемнику удалось верно преобразовать семплы в текст, значит, удалось верно реализовать логику работы приемника.

ВЫВОД

В ходе проделанной работы я реализовал логику работы применой стороны на языке C/C++. Выяснил, как выбор семпла в качестве значения символа влияет на созвездие. Убедился в необходимости символьной синхронизации.